Cloud Computing

Lecture 2 - Enabling Technologies - Containerization

Emiliano Casalicchio, Phd
Department of Computer Science
Sapienza University of Rome

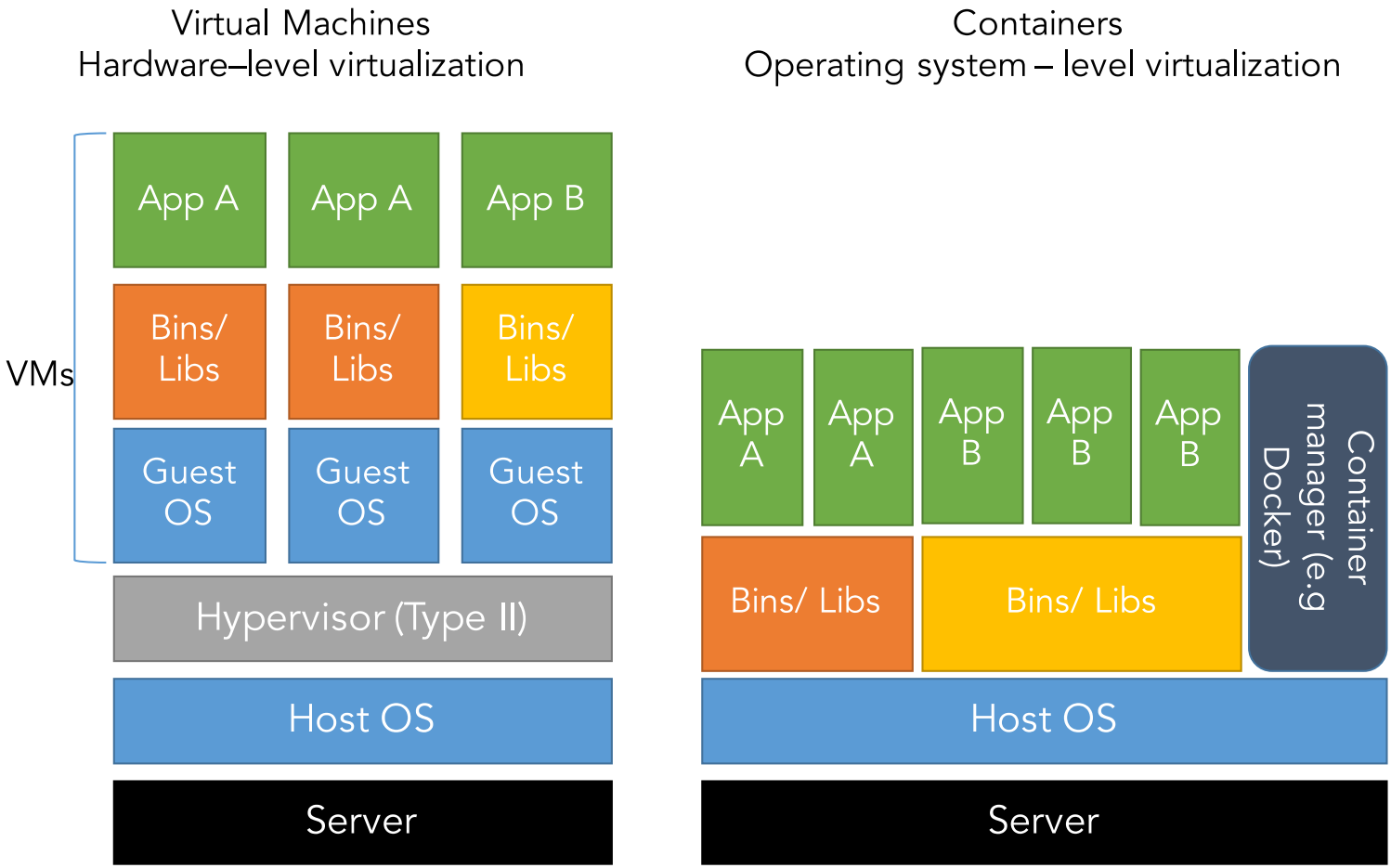
Agenda

- The slides
- Links provided in the slides

- Containerization
  - What is a container
  - Fundamental tech for container
  - Docker

Operating system level virtualization: Containerization

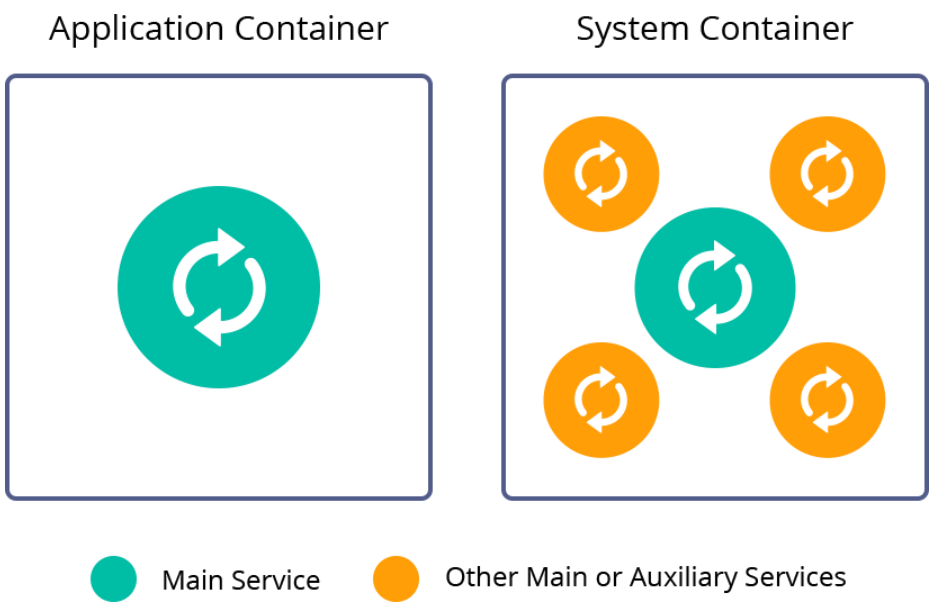
- **Containerization** – leverage multiprogramming techniques at OS level
- No virtual machine manager or hypervisor
- No HW emulation, sharing of the same host OS
- Based on
  - **Namespace** - what you can see and use
  - **Cgroups** - how much you can use
  - **unionFS** - to marge fs



Application Containers vs System Containers

- A container can be used to distribute and run a single application, a microservice, or an entire operating systems

Application containers	System containers
Contains a single process	Contains a entire Operating System
Use a layerd filesystem	File system neutral
Run microservices / application	Run a lightweight virtual machine
Used for distributing applications	Used for providing underlining infrastructure
Implementations: Docker, CRI-O	Implementations: LXC/LXD, OpenVZ/Virtuozzo, BSD jails, Linux vServer,



Containerization

- An evolution of the *chroot* mechanism
  - the *chroot* operation changes the file system root directory for a process and its children to a specific directory
  - the process and its children cannot have access to other portions of the file system than those accessible under the new root directory
  - unix systems also expose devices as parts of the file system, by using this method it is possible to completely isolate a set of processes

- Nowadays chroot is not used by container runtimes any more and was replaced by **pivot_root(2)**

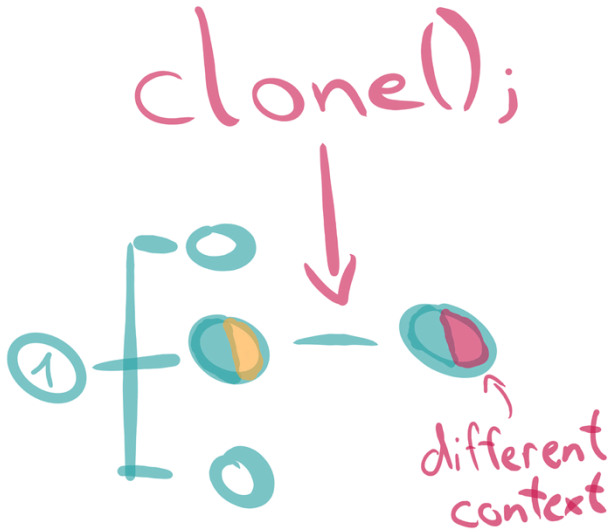
Namespace

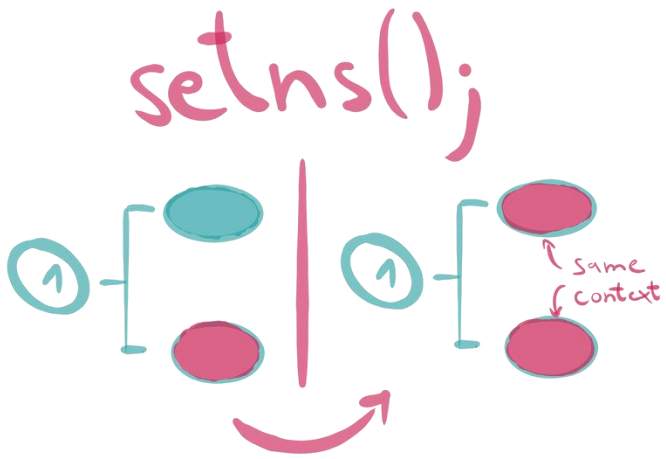
http://man7.org/linux/man-pages/man7/namespaces.7.html

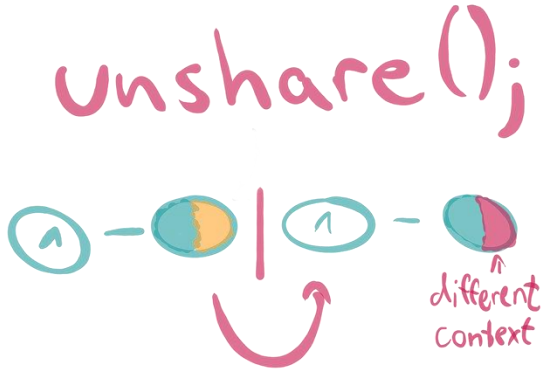
- Wraps a global system resource in an abstraction
  - **Cgroup** Cgroup root directory
  - **IPC** System V IPC, POSIX message queues
  - **Network** Network devices, stacks, ports, etc.
  - **Mount** Mount points
  - **PID** Process IDs
  - **User** User and group IDs
  - **UTS** Hostname and NIS domain name
- The processes within the namespace perceive they have their own isolated instance of the global resource
- Changes to the resource in the name space are
  - visible to other processes members of the namespace
  - invisible to other processes.

Namespace

- At operating system level, when a process is created with the clone() system call one or more namespace can be created.
- An argument of the clone() system call allow to specify what namespace to create
- There are other system call that allow a process to join an existing name space (setns) and to move a process's context in a new namespace (un-share)







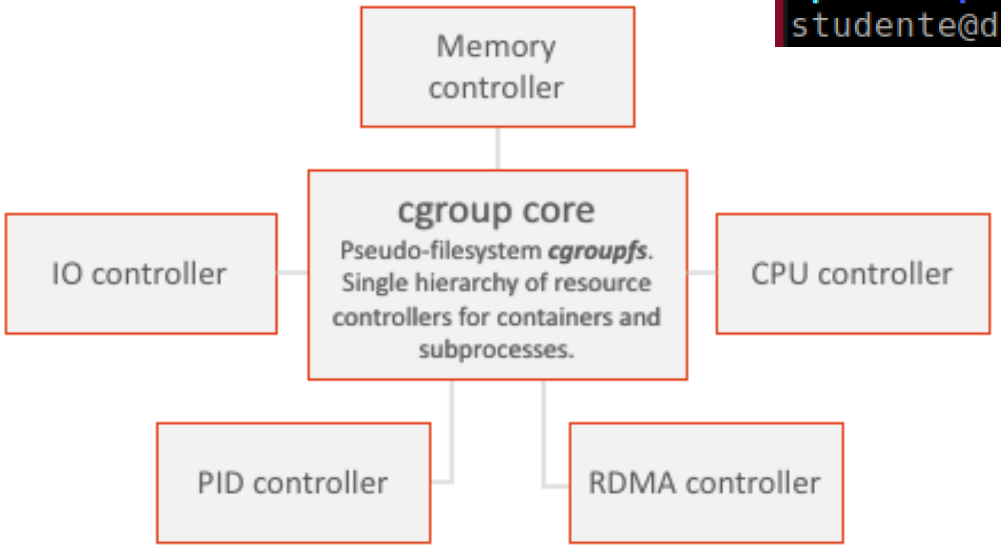
https://medium.com/@saschagrunert/demystifying-containers-part-i-kernel-space-2c53d6979504

Cgroups

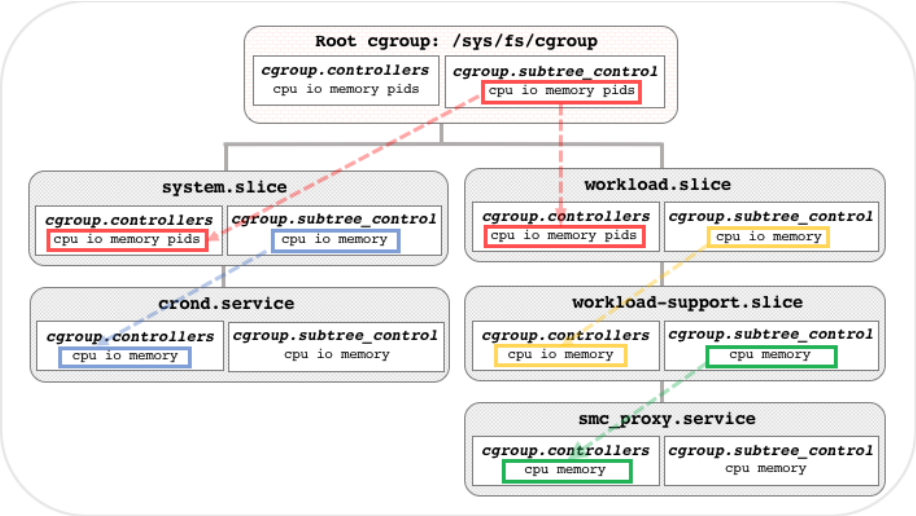
http://man7.org/linux/man-pages/man7/cgroups.7.html

- The main goal of Control groups (cgroups) is to support resource limiting, prioritization, accounting and controlling
  - Allow processes to be organized into hierarchical groups
- The kernel's cgroup interface is provided through a pseudo-filesystem called cgroupfs.

```
studente@debian9:/sys/fs/cgroup$ ls
blkio  cpuacct      cpuset   freezer  net_cls          net_prio    pids
cpu    cpu,cpuacct  devices  memory   net_cls,net_prio perf_event  systemd
studente@debian9:/sys/fs/cgroup$
```

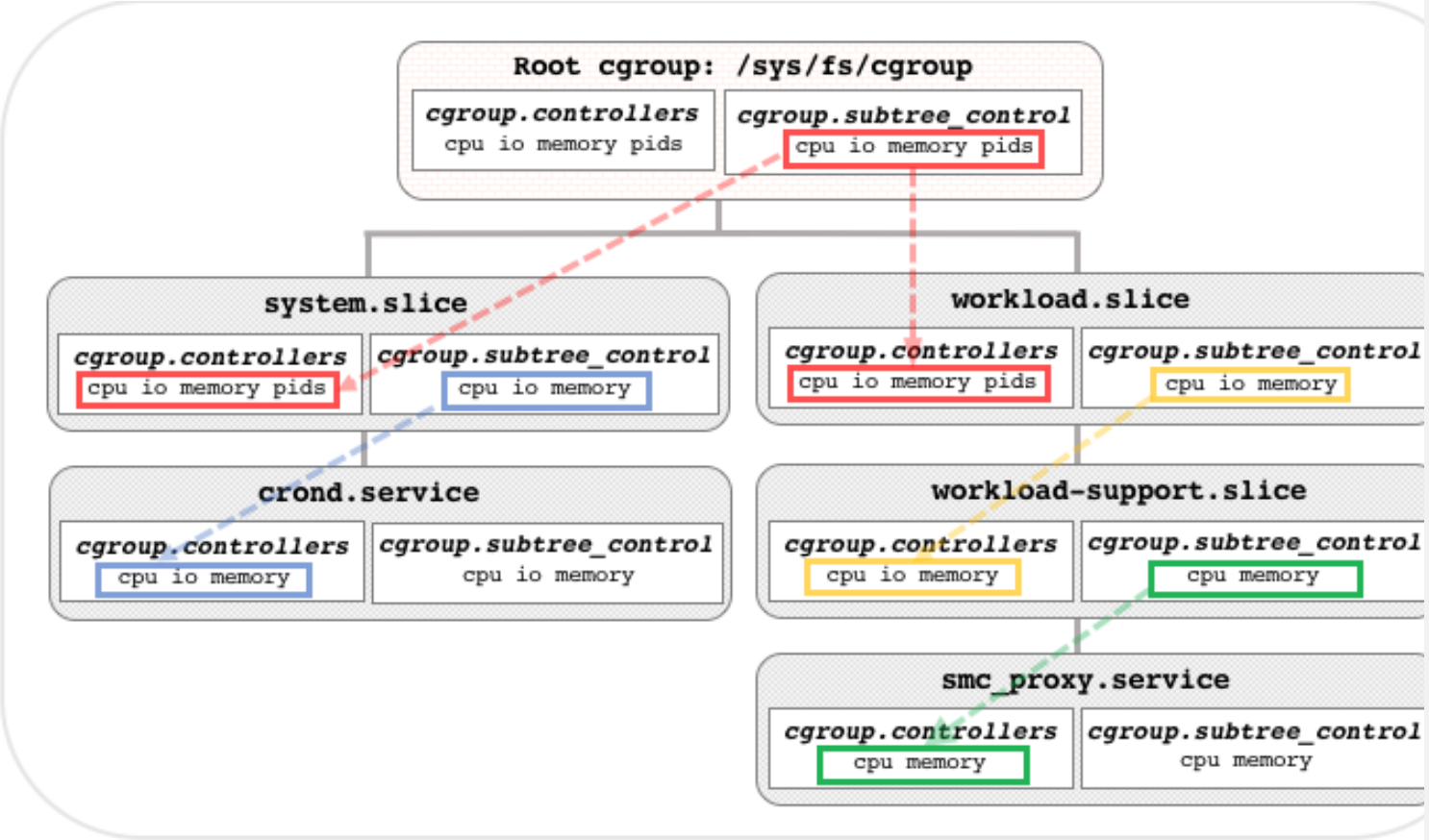


Cgroup




```
studente@debian9:/sys/fs/cgroup/memory$ ls
cgroup.clone_children                 memory.kmem.usage_in_bytes
cgroup.event_control                  memory.limit_in_bytes
cgroup.procs                          memory.max_usage_in_bytes
cgroup.sane_behavior                  memory.move_charge_at_immigrate
demo                                  memory.numa_stat
memory.failcnt                        memory.oom_control
memory.force_empty                    memory.pressure_level
memory.kmem.failcnt                   memory.soft_limit_in_bytes
memory.kmem.limit_in_bytes            memory.stat
memory.kmem.max_usage_in_bytes        memory.swappiness
memory.kmem.slabinfo                  memory.usage_in_bytes
memory.kmem.tcp.failcnt               memory.use_hierarchy
memory.kmem.tcp.limit_in_bytes        notify_on_release
memory.kmem.tcp.max_usage_in_bytes    release_agent
memory.kmem.tcp.usage_in_bytes        tasks
studente@debian9:/sys/fs/cgroup/memory$ cd demo
studente@debian9:/sys/fs/cgroup/memory/demo$ ls
cgroup.clone_children                 memory.limit_in_bytes
cgroup.event_control                  memory.max_usage_in_bytes
cgroup.procs                          memory.move_charge_at_immigrate
memory.failcnt                        memory.numa_stat
memory.force_empty                    memory.oom_control
memory.kmem.failcnt                   memory.pressure_level
memory.kmem.limit_in_bytes            memory.soft_limit_in_bytes
memory.kmem.max_usage_in_bytes        memory.stat
memory.kmem.slabinfo                  memory.swappiness
memory.kmem.tcp.failcnt               memory.usage_in_bytes
memory.kmem.tcp.limit_in_bytes        memory.use_hierarchy
memory.kmem.tcp.max_usage_in_bytes    notify_on_release
memory.kmem.tcp.usage_in_bytes        tasks
memory.kmem.usage_in_bytes
studente@debian9:/sys/fs/cgroup/memory/demo$
```

Cgroup



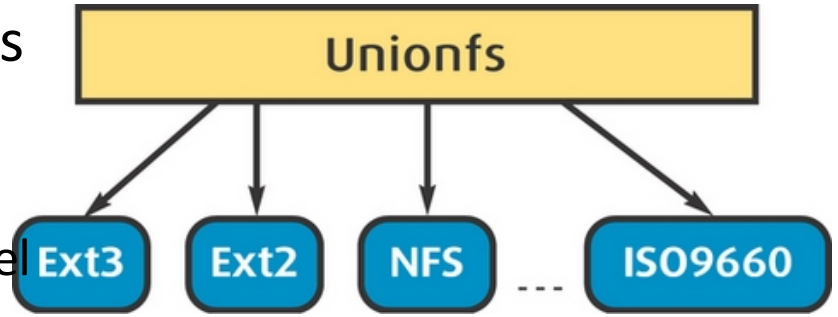

```
> sudo mkdir /sys/fs/cgroup/memory/demo
> ls /sys/fs/cgroup/memory/demo
cgroup.clone_children
cgroup.event_control
cgroup.procs
memory.failcnt
memory.force_empty
memory.kmem.failcnt
memory.kmem.limit_in_bytes
memory.kmem.max_usage_in_bytes
memory.kmem.slabinfo
memory.kmem.tcp.failcnt
memory.kmem.tcp.limit_in_bytes
memory.kmem.tcp.max_usage_in_bytes
memory.kmem.tcp.usage_in_bytes
memory.kmem.usage_in_bytes
memory.limit_in_bytes
memory.max_usage_in_bytes
memory.move_charge_at_immigrate
memory.numa_stat
memory.oom_control
memory.pressure_level
memory.soft_limit_in_bytes
memory.stat
memory.swappiness
memory.usage_in_bytes
memory.use_hierarchy
notify_on_release
tasks
```


Emiliano.Casalicchio@bth.se

UnionFS

Charles P. Wright (2004) Unionfs: Bringing Filesystems Together
https://www.linuxjournal.com/article/7714

- Unioning allows administrators to keep files separate physically, but to merge them logically into a single view
- Union: a collection of merged directories; each physical directory is called a branch
- UnionFS simultaneously layers on top of several filesystems or directories
  - presents a filesystem interface to the kernel
  - it can be employed by any user-level application or from the kernel by the NFS server
  - intercepts operations bound for lower-level filesystems, it can modify operations to present the unified view



```
$ ls /Fruits
Apple Tomato
$ ls /Vegetables
Carrots Tomato
$ cat /Fruits/Tomato
I am botanically a fruit.
$ cat /Vegetables/Tomato
I am horticulturally a vegetable.
# mount -t unionfs -o dirs=/Fruits:/Vegetables none
/mnt/healthy
$ ls /mnt/healthy
Apple Carrots Tomato
$ cat /mnt/healthy/Tomato
I am botanically a fruit.
```

%%% here Fruits has higher priority than Vegatables

Union FS (cont'd)

- To each branch is assigned a precedence
- A branch with a higher precedence overrides a branch with a lower precedence.
- Unionfs operates on directories
  - If a directory exists in two underlying branches, the contents and attributes of the Unionfs directory are the combination of the two lower directories
- Unionfs automatically removes any duplicate directory entries
  - If a file exists in two branches, the contents and attributes of the Unionfs file are the same as the file in the higher-priority branch, and the file in the lower-priority branch is ignored

(Read example about /Fruit and /Vegetable in Charles P. Wright)

Union FS: Copy-on-Write Unions

- Unionfs also can mix read-only and read-write branches.
- In this case, **the union as a whole is read-write**
- *copy-on-write* semantics **to give the illusion that you can modify files and directories on read-only branches**
- Example
  - /mnt/cdrom is read only
  - /tmp/cdpatch is read and write (created by the admin)

```
# mount -t unionfs -o dirs=/tmp/cdpatch,/mnt/cdrom none /mnt/patched-cdrom
```

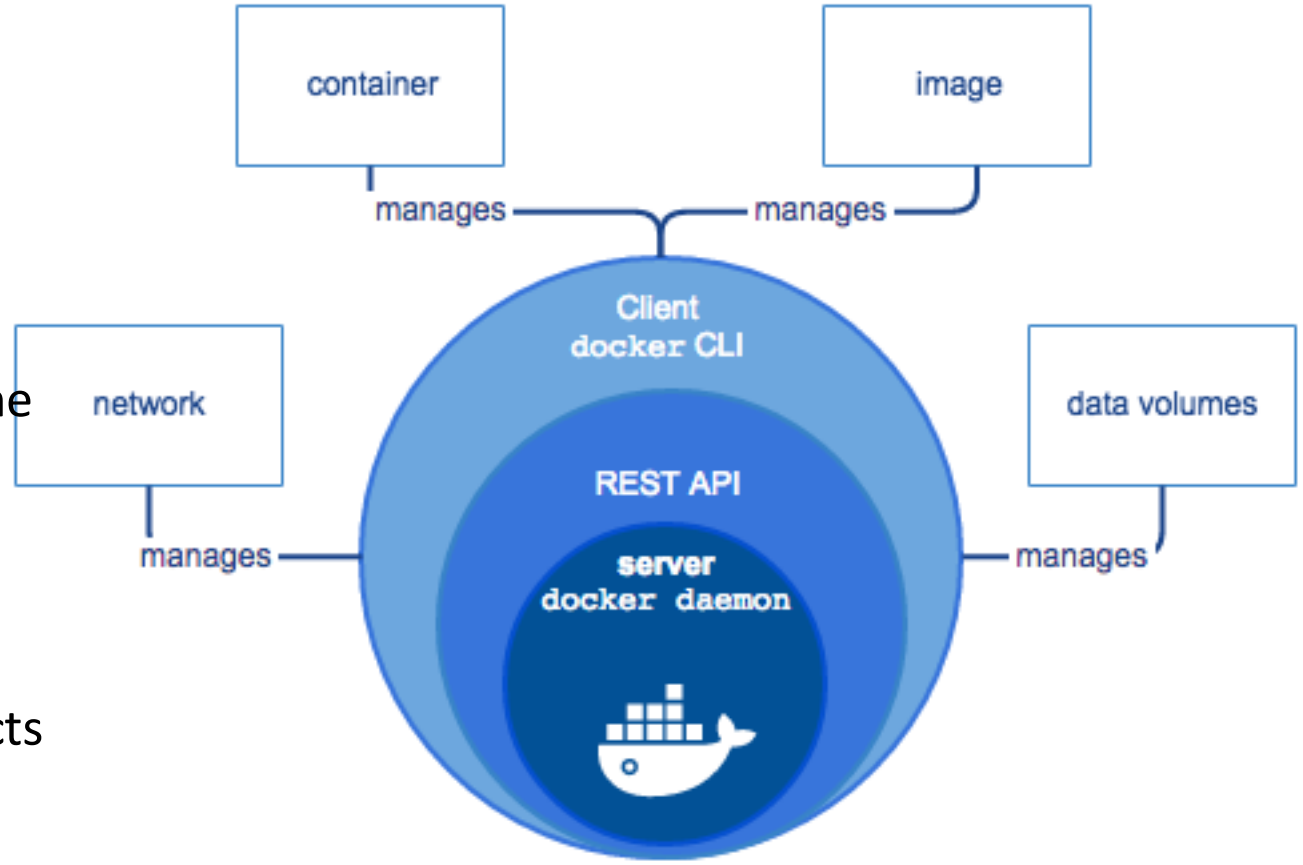
- Through /mnt/patched-cdrom, it appears you can write to the **`/mnt/cdrom`**
  - all writes actually will take place in /tmp/cdpatch.
  - Writing to read-only branches results in an operation called a **copyup**
  - When a read-only file is opened for writing, the file is copied over to a higher-priority branch. If required, Unionfs automatically creates any needed parent directory hierarchy.

Containerization in practice

Docker

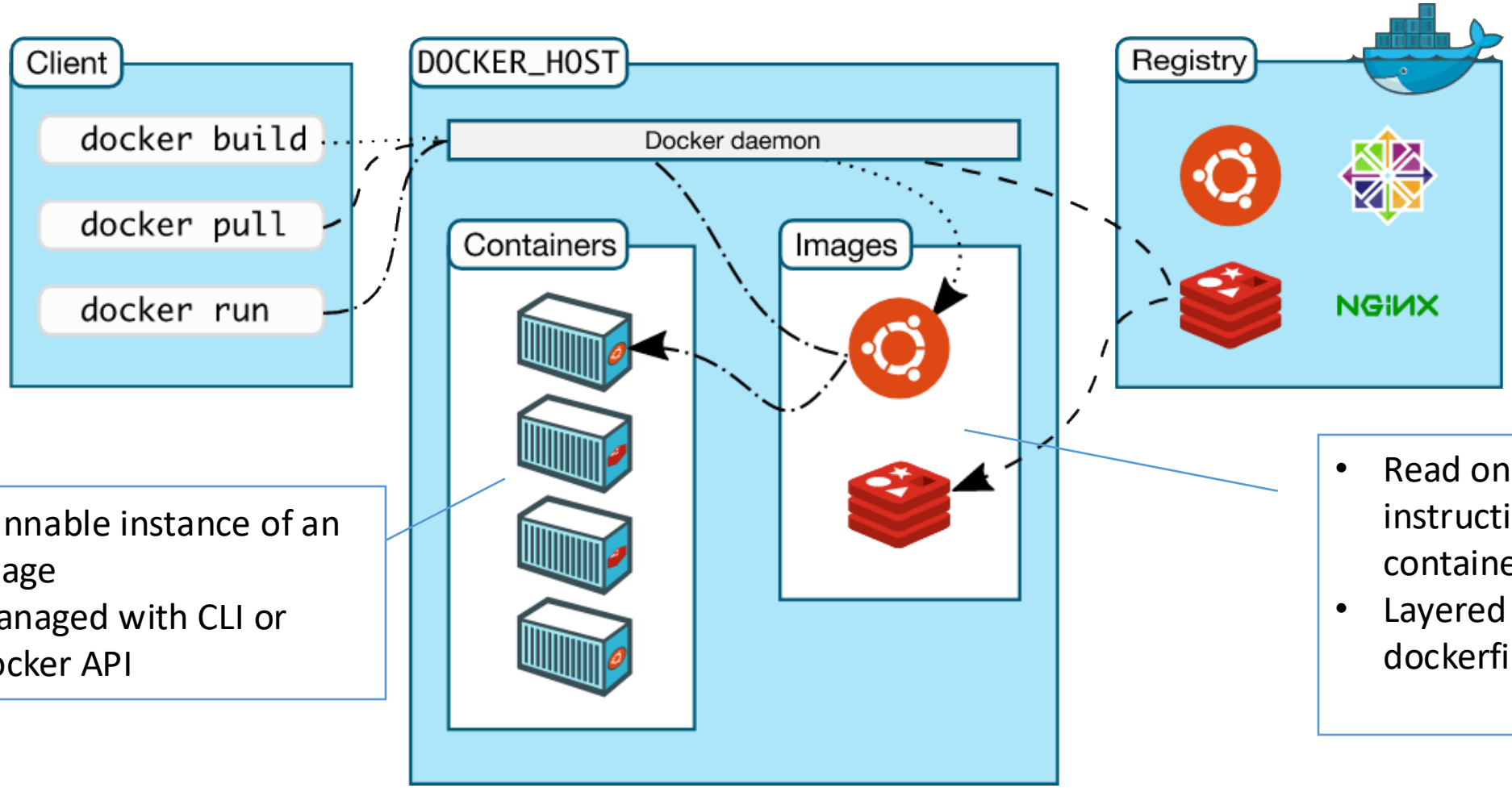
Containerization with Docker

- Containers run directly within the host machine's kernel
  - Containers + bare metal
  - Containers + virtual machine
- Docker engine is a C/S application
  - CLI command line interface to interact with the server (Docker daemon)
  - CLI use the Docker REST API
  - Docker REST API can be used also separately (called from applications)
  - The daemon create and manage Docker objects



Courtesy by docker.com

Docker architecture

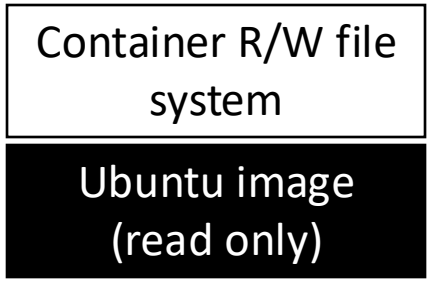


- Runnable instance of an image
- Managed with CLI or Docker API

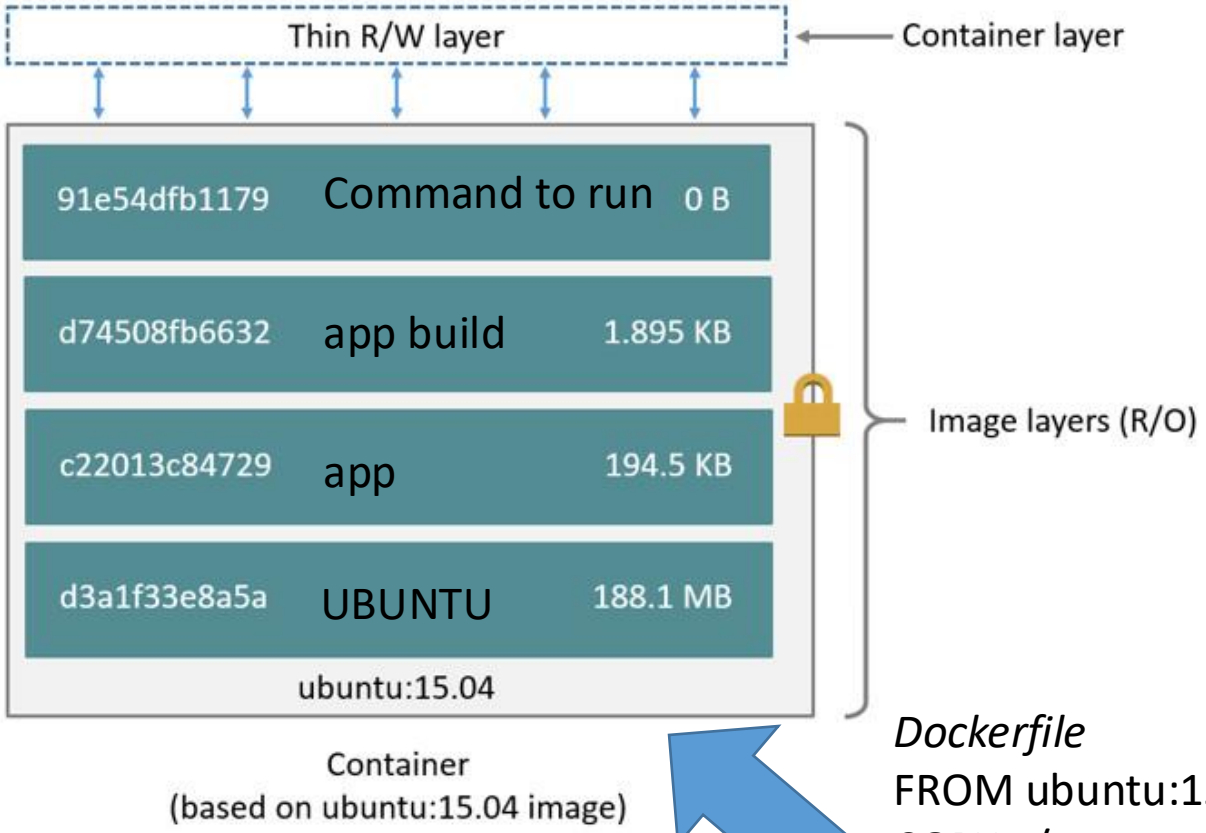
- Read only template with instruction for creating a container
- Layered (defined by a dockerfiles)

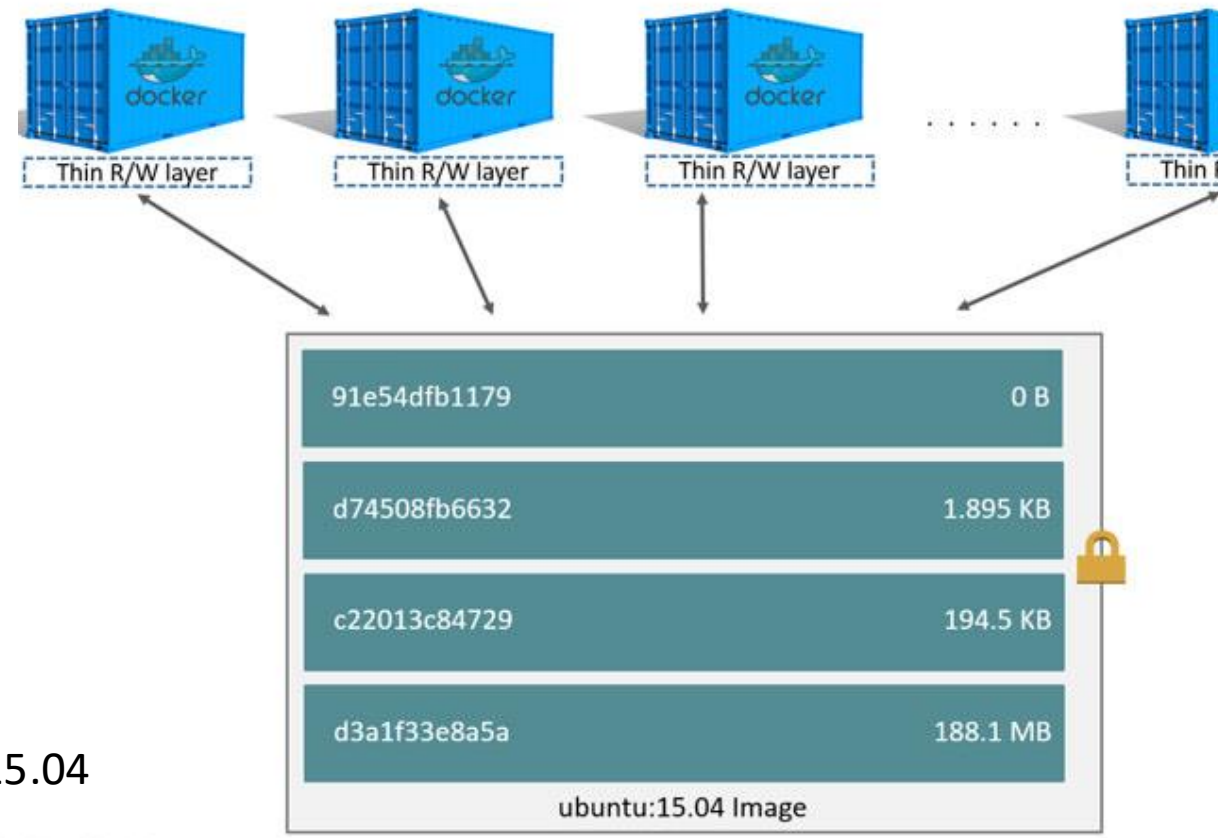
Example of a docker CLI command

- `docker run -i -t ubuntu /bin/bash`
- The Docker deamon does what follow
  - Locate and eventually download the ubuntu image (if not locally stored)
  - Create a new container
  - Allocate a R/W file system as final layer of the image
    - The file system is isolated from the host file system
  - Create a network interface connected to the default network
  - Start the container and run /bin/bash command



Layers concept





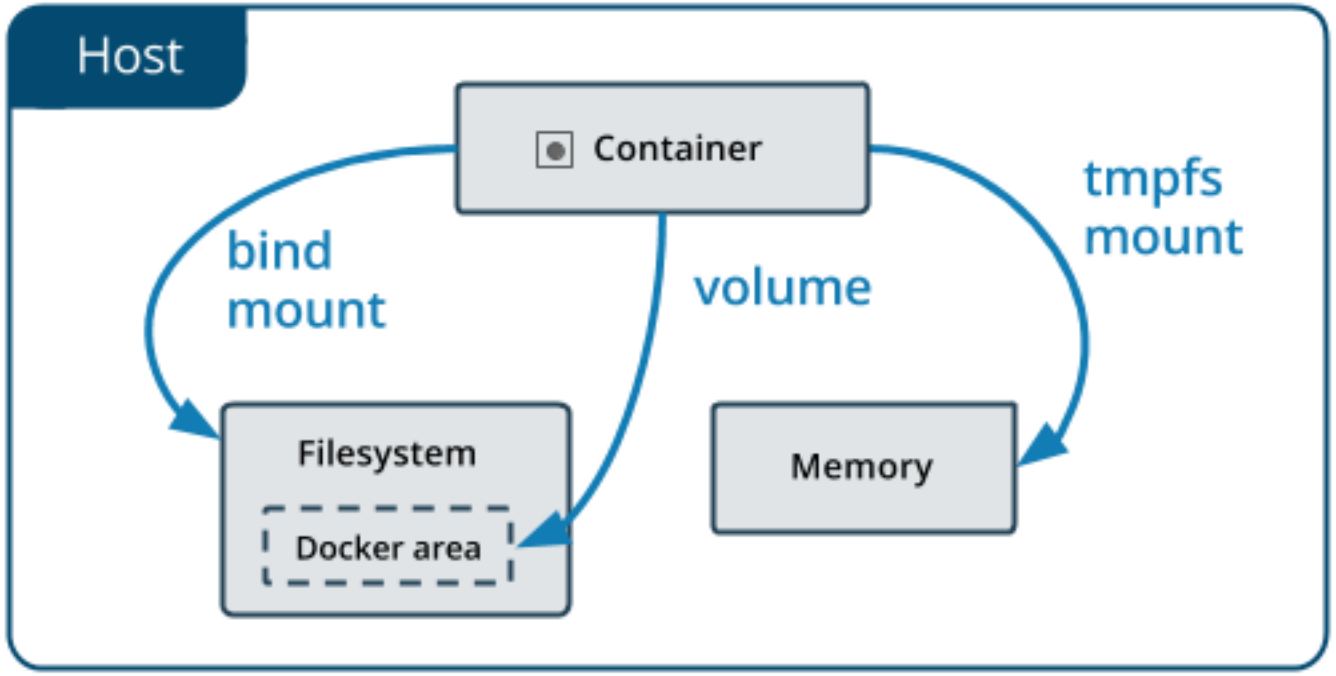
Dockerfile

```
FROM ubuntu:15.04
COPY . /app
RUN make /app
CMD python /app/app.py
```

Storage options

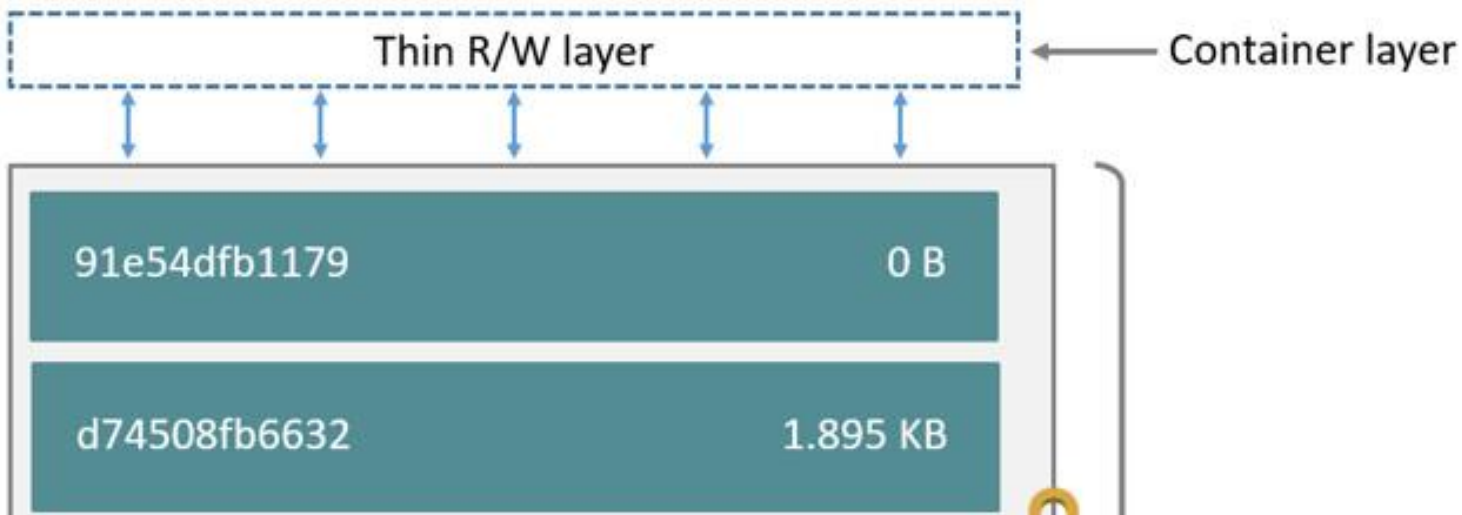
- Container writable layer
- Bind mount
- Volumes
- tmpfs mount

REFERENCE
https://docs.docker.com/storage/



Storage - Container writable layer

- Does not persist when the container is terminated
- Writable layer is tightly coupled with the host where the container is running – reduced portability
- Reduced performance



Storage - Volume

- Created and managed by docker (isolated from the host)
- Stored within a directory on the Docker host
- Mounted into multiple containers simultaneously R/W or R
- Advantages
  - Easier to back up or migrate than bind mounts.
  - Volumes can be more safely shared among multiple containers.
  - Volume drivers let you store volumes on remote hosts or cloud providers, to encrypt the contents of volumes, or to add other functionality.
  - New volumes can have their content pre-populated by a container.
  - Volumes work on both Linux and Windows containers.
  - Volumes can be managed using Docker CLI commands or the Docker API (you cannot access directly data in a volume like a normal filesystem).

Storage - bind mount

- Any area of the host file system
- A file or directory on the *host machine* is mounted into a container
  - referenced by its full path on the host machine
  - if not exist created by the container
- Shared with host processes
- Not portable - depends on the filesystem
- Performance usually higher than Volumes

Storage - tmpfs

- An area of memory outside the container writable layer
- A memory area in the container namespace or shared with the host
- Is temporary, and only persisted in the host memory
  - Removed when the container stop
- Limitations
  - Available only in Linux hosts
  - Not sharable among containers

Activity: Docker storage options at work (5 min)

- To back up, restore, or migrate data from one Docker host to another
- To share data among multiple running containers
- To share configuration files from the host machine to containers
- Data shouldn't persist either on the host machine or within the container e.g. for security reasons, or to manipulate large volume of non-persistent data with high performance
- To share source code or to build artifacts between a development environment on the Docker host and a container
- No guarantee to have a given directory or file structure in the Docker host
- To store container's data on a remote host or a cloud provider, rather than locally
- The file or directory structure of the Docker host is guaranteed to be consistent with the one the containers require

- Volumes
- Bind mount
- Tmpfs mount

Networking

- You can connect Container together, or connect them to non-Docker workloads
  - Docker containers do not even need to be aware that they are deployed on Docker, or whether their peers are also Docker workloads or not
- Type of network drivers
  - Bridge
  - Host
  - Overlay
  - Macvlan
  - none

Network drivers

- **bridge**:
  - The default network driver
  - If you don’t specify a driver, this is the type of network you are creating
  - Bridge networks are usually used when your applications run in standalone containers that need to communicate
- **host**:
  - For standalone containers
  - Remove network isolation between the container and the Docker host, and use the host’s networking directly

Network drivers (cont'd)

- **overlay**:
  - Overlay networks connect multiple Docker daemons together and enable swarm services to communicate with each other
  - You can also use overlay networks to facilitate communication between a swarm service and a standalone container, or between two standalone containers on different Docker daemons
  - This strategy removes the need to do OS-level routing between these containers.

Network drivers (cont'd)

- **macvlan:**
  - Macvlan networks allow you to assign a MAC address to a container, making it appear as a physical device on your network
  - The Docker daemon routes traffic to containers by their MAC addresses
  - Using the macvlan driver is sometimes the best choice when dealing with legacy applications that expect to be directly connected to the physical network, rather than routed through the Docker host's network stack.
- **none:**
  - For this container, disable all networking

Activity: Networking Drivers cases

- **Y**ou are migrating from a VM setup
- **T**he network stack should not be isolated from the Docker host
- **C**ontainers running on different Docker hosts should communicate
- Multiple containers should communicate on the same Docker host.
- You need your containers to look like physical hosts on your network (a unique MAC address).

- **User-defined bridge networks**
- **Host networks**
- **Overlay networks**
- **Macvlan networks**

Outcome

- Docker and its underlining technologies
  - Namespace and cgroup
  - Union FS
  - Roles of Docker deamon, CLI and API
  - Layerd images
  - Storage: volume, bind, tmpfs
  - Networking

Questions?